PROVISIONAL PATENT APPLICATION

MULTI-DIMENSIONAL AUTONOMOUS UNIVERSAL LAW DISCOVERY SYSTEM WITH MATHEMATICAL FIELD INTERACTION ALGORITHMS

VALHALLA PROJECT

Inventor: [YOUR NAME]

Filing Date: May 28, 2025

Attorney Docket No: VALHALLA-001

CORE MATHEMATICAL ALGORITHMS

8-Dimensional Natural Field Interaction Algorithm

$$I(A,B) = \sum_{i=1}^{8} [A_i \times B_i] \times \sqrt{\rho_A \times \rho_B} \times \frac{\phi_A + \phi_B}{2} \times (1 - |r_A - r_B|)$$

Threshold-Free Fusion Probability Algorithm

$$P_{fusion} = \frac{1}{1 + e^{-3.0 \times I(A,B) \times \sqrt{V_A \times V_B}}}$$

Dynamic Threshold Calculation Method

$$T_{dynamic} = \mu + \beta \times \sigma \times (1 + \gamma \times \ln(n))$$

$$R_i = R_{total} \times \frac{W_i \times P_i \times U_i}{\sum_{j=1}^{4} [W_j \times P_j \times U_j]}$$

FIELD OF THE INVENTION

This invention relates to autonomous artificial intelligence systems for scientific discovery, specifically to mathematical algorithms for multi-dimensional field interactions, quaternary physical system coordination, and threshold-free decision making in unlimited computational environments.

BACKGROUND OF THE INVENTION

Current artificial intelligence systems for scientific discovery are limited by:

1. Fixed mathematical thresholds that prevent adaptive behavior

2. Limited dimensional exploration due to computational constraints

3. Inability to process multiple physical phenomena simultaneously

4. Lack of natural field interaction mathematical models

5. Resource limitations that cap discovery potential

No existing system provides mathematical algorithms for threshold-free natural interactions combined with unlimited resource management for autonomous scientific discovery.

SUMMARY OF THE INVENTION

The present invention provides specific mathematical algorithms and computational methods comprising:

1. **8-Dimensional Natural Field Interaction Algorithm** with mathematical formula:

```
I(A,B) = Σ(i=1 to 8)[A_i × B_i] × √(ρ_A × ρ_B) × ((φ_A + φ_B)/2) × (1 - |r_A - r_B|)
```

Where I(A,B) is interaction strength, A_i and B_i are field vector components, ρ is density, φ is fluidity, and r is resonance.

2. **Threshold-Free Fusion Probability Algorithm**:

```
P_fusion = 1 / (1 + e^(-α × I(A,B) × √(V_A × V_B)))
```

Where α is activation coefficient (3.0), and V is vitality factor.

3. **Dynamic Threshold Calculation Method**:

$$T_{dynamic} = \mu + \beta \times \sigma \times (1 + \gamma \times \log(n))$$

Where μ is mean distance, σ is standard deviation, β is adaptive factor, γ is complexity coefficient, and n is data points.

4. **Quaternary System Resource Allocation Algorithm**:

DETAILED MATHEMATICAL DESCRIPTION

8-Dimensional Natural Field Interaction Algorithm

Field Vector Definition:

Each natural cell maintains an 8-dimensional force field vector:

F = [f_density, f_fluidity, f_resonance, f_emergence, f_complexity,
f_adaptation, f_coherence, f_potential]

Natural Interaction Calculation:

The interaction strength between two cells A and B is calculated using the proprietary formula:

```
I(A,B) = Dot_Product(F_A, F_B) × Density_Factor × Fluidity_Factor × Resonance_Factor
```

Where:

- `Dot_Product(F_A, F_B) = Σ(i=1 to 8)[F_A[i] × F_B[i]]`

- `Density_Factor = √(ρ_A × ρ_B)`

- `Fluidity_Factor = (φ_A + φ_B) / 2`

- `Resonance_Factor = 1 - |r_A - r_B|`

Threshold-Free Fusion Probability Algorithm

Sigmoid Activation Function:

The fusion probability is calculated using a continuous sigmoid function:

```
P_fusion(A, B) = 1 / (1 + exp(-activation_coefficient × interaction_strength × vitality_product))
```

Where:

- `activation_coefficient = 3.0` (optimized through testing)

- `interaction_strength = I(A,B)` (from above formula)

- `vitality_product = √(V_A × V_B)`

Implementation Algorithm:

fn calculate_fusion_probability(cell_a: &NaturalCell, cell_b: &NaturalCell) ->
f64 {

let interaction = calculate_field_interaction(&cell_a.field, &cell_b.field);

let vitality_factor = (cell_a.vitality * cell_b.vitality).sqrt();

let raw_probability = interaction * vitality_factor;

Dynamic Threshold Calculation Method

Statistical Distribution Analysis:

Dynamic thresholds are calculated using statistical properties of the data distribution:

T_dynamic = μ_distance + β_adaptive × σ_distance × (1 + γ_complexity × ln(n_points))

Where:

- `μ_distance` = mean distance between all data points

- `σ_distance` = standard deviation of distances

- `β_adaptive` = adaptive scaling factor (typically 0.5-1.5)

- `γ_complexity` = complexity coefficient (0.1-0.3)

- `n_points` = number of data points

Implementation Algorithm:

fn calculate_dynamic_threshold(data_points: &[HyperPoint]) -> f64 {

Quaternary System Resource Allocation Algorithm

Infinite Resource Management:

Resources are allocated among the four systems using the weighted priority algorithm:

R_BlackHole = R_total × (W_BH × P_BH × U_BH) / Σ_all_systems

R_MeshBrain = R_total × (W_MB × P_MB × U_MB) / Σ_all_systems

R_MultiNova = R_total × (W_MN × P_MN × U_MN) / Σ_all_systems

R_BaseForce = R_total × (W_BF × P_BF × U_BF) / Σ_all_systems

Where:

- `W_i` = system weight (based on current task requirements)

- `P_i` = priority level (1.0-10.0 scale)

- `U_i` = current utilization efficiency (0.0-1.0)

Dynamic Weight Calculation:

AUTONOMOUS LAW DISCOVERY MATHEMATICAL PROCESS

Step 1: Multi-System Parallel Analysis

Each system processes input data using specialized mathematical models:

Result_i = System_i.process(input_data, mathematical_model_i)

Step 2: Cross-System Validation

Validation score between systems i and j:

V(i,j) = correlation(Result_i, Result_j) × confidence_factor

Step 3: Unified Synthesis

Combined result using weighted average:

Final_Result = Σ(i=1 to 4)[W_i × Result_i × Confidence_i] / Σ(i=1 to 4)[W_i × Confidence_i]

Step 4: Pattern Recognition Algorithm

fn discover_mathematical_patterns(data: &[f64], min_pattern_length: usize) -> Vec<Pattern> {

CLAIMS

1. A mathematical algorithm for calculating natural field interactions comprising:

- An 8-dimensional force field vector representation

- A dot product calculation combined with density, fluidity, and resonance factors

- The specific formula: $I(A,B) = \Sigma(i=1 \text{ to } 8)[A_i \times B_i] \times \sqrt{(\rho_A \times \rho_B)} \times ((\varphi_A + \varphi_B)/2) \times (1 - |r_A - r_B|)$

2. A threshold-free fusion probability algorithm comprising:

- A sigmoid activation function with coefficient 3.0

- The specific formula: $P_{fusion} = 1 / (1 + e^{(-3.0 \times I(A,B) \times \sqrt{(V_A \times V_B)})})$

- Continuous probability calculation without discrete cutoff points

3. A dynamic threshold calculation method comprising:

- Statistical analysis of data distribution properties

MATHEMATICAL IMPLEMENTATION DETAILS

Gaussian Noise Generation:

fn generate_gaussian_noise(mean: f64, std_dev: f64) -> f64 {

use rand_distr::{Normal, Distribution};

let normal = Normal::new(mean, std_dev).unwrap();

normal.sample(&mut rand::thread_rng())

}

Pairwise Distance Calculation:

fn calculate_all_pairwise_distances(points: &[HyperPoint]) -> Vec<f64> {

let mut distances = Vec::new();

for i in 0..points.len() {

for j in i+1..points.len() {

let distance = euclidean_distance(&points[i], &points[j]);

ABSTRACT

A mathematical algorithm system for autonomous universal law discovery comprising specific formulas for 8-dimensional natural field interactions, threshold-free fusion probability calculation using sigmoid activation, dynamic threshold computation based on statistical distribution analysis, and quaternary system resource allocation. The system uses proprietary mathematical models including $I(A,B) = \Sigma(i=1 \text{ to } 8)[A_i \times B_i] \times \sqrt{(\rho_A \times \rho_B)} \times ((\varphi_A + \varphi_B)/2) \times (1 - |r_A - r_B|)$ for field interactions and $P_{fusion} = 1 / (1 + e^{(-3.0 \times I(A,B) \times \sqrt{(V_A \times V_B)})})$ for fusion probability, enabling autonomous discovery of physical laws without human intervention or predetermined constraints.